

Data Structures and Algorithms (23CSE203)

Lab Manual

Submitted By

NAME	CH. JAI RAM CHARAN
ROLL NO	AV.SC.U4CSE25106
YEAR/SEM/SECTION	2ND YEAR/ SEM-3/CSE-B
CREDITS	05

Submitted to

NAME	DR. RAJ KUMAR BATCHU
DESIGNATION	ASST.PROFFESSOR (Sl. Grade)
DEPARTMENT	SCHOOL OF COMPUTING

Index

S.NO	TITLE / TOPICS	DATE
1	**WEEK 1:** **Recursion** A) Recursive program for countdown from n to 1, followed by 'launch'. B) Recursive program to calculate P^n (compound interest growth factor P, years n). C) Recursive program to print first n terms of Fibonacci series. D) Recursive program to search for an employee ID in a list. E) Recursive program to calculate factorial (n!) for parcel arrangement.	14-07-26
2	**WEEK 2:** **Searching** A) Python program to perform Linear Search on given input. B) Python program to perform Binary Search on given input (verify with sorted and unsorted lists).	21-07-26
3	**WEEK 3:** **Elementary Sorting** A) Python program to implement Bubble Sort (user input). B) Python program to implement Selection Sort (user input). C) Python program to implement Insertion Sort (user input).	01-08-26
4	**WEEK 4:** **Advanced Sorting** A) Python program to implement Merge Sort (user input). B) Python program to implement Quick Sort (user input).	11-08-26
5	**WEEK 5:** **Linked Lists** Implement operations for: A) Singly Linked List (SLL) B) Doubly Linked List (DLL) C) Circular Linked List (CLL) Operations: 1. Create LL 2. Insert at Beginning 3. Insert at End 4. Insert at Index 5. Delete by Value 6. Delete Prior Node 7. Delete Last Node 8. Count Nodes 9. Display/Traverse 10. Exit	18-08-26
6	**WEEK 6:** **Stacks** A) Implement Stack operations (Push, Pop, Peek, Display) using Array.	25-08-26

	B) Implement Stack operations (Push, Pop, Peek, Display) using Linked List.	
7	**WEEK 7:** **Queues** A) Implement Queue using Arrays. B) Implement Queue using Linked List. C) Implement Circular Queue using Arrays. D) Implement Circular Queue using Linked List.	22-09-26
8	**WEEK 8:** **Binary Trees** A) Implement Binary Tree with operations using Array. B) Implement Binary Tree with operations using Linked List.	29-09-26
9	**WEEK 9:** **Binary Search Trees** Python program to implement Binary Search Tree (BST) operations.	06-10-26
10	**WEEK 9:** **AVL Trees** Python program to implement AVL Tree with all it's operations.	13-10-26
11	**WEEK 11:** **Hashing** Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).	27-10-26

Lab – 1

1a)

Aim:

- A rocket launch system displays a count down before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display count down from n to 1 followed by launch.

Program:

```python
#Count Down #CSE25106
def countdown(n):
    if n >= 0 :
        if n != 0:
            print(n)
            return countdown(n-1)
        else:
            print("Launch!")

n= int(input("Enter the number:"))
countdown(n)
```

Output:

```
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\
Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_1.py
Enter the number:10
10
9
8
7
6
5
4
3
2
1
Launch!
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA>
```

Error Table:

Error	Rectification
Output: None after launch	Swapping if and else case.

Description:

- The program displays a countdown from a given number n to 1.

- It uses a recursive function named countdown().
- The function calls itself with the value n-1.
- The recursion continues until n becomes 0.
- When n reaches 0, the program displays Launch.

1b)

Aim:

➢ Banks Calculate the growth of investment using compound interest. The formula requires calculating powers , which can be implemented recursively. Write a recursive program to calculate P^n where P = Principal growth amount, n = no of years

Program:

```
#Growth #CSE25106
n = int(input("Enter number of years:"))
p = int(input("Enter annual principal:"))
def growth(p,n):
    if n < 0:
        return "Invalid input for number of years"
    if n == 0:
        return 1
    else:
        return p*growth(p,n-1)

print(growth(p,n))
```

Output:

```
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\
Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_2.py
Enter number of years:1
Enter annual principal:300
300
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\
Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_2.py
Enter number of years:-6
Enter annual principal:66
Invalid input for number of years
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA>
```

Error Table:

Error	Rectification
Recursion Error: maximum recursion depth exceeded	Added if case to give valid input for n.

Description:

- P represents the principal/growth amount.
- n represents the number of years.
- The user enters the values of P and n.
- The growth() function calls itself with n-1.
- When n becomes 0, the function returns 1.
- The result is calculated by repeatedly multiplying P.

1c)

Aim:

- A HR department stores employee details in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

Program:

```
#Searching of element #CSE25106
list = []
t=int(input("Enter number of inputs:"))
for _ in range(t):
    n = int(input("Enter the ID:"))
    list.append(n)
ID = int(input("Enter ID to seach:"))
start = 0
def search(list,ID,start):
    if list[start] == ID:
        print(f"ID found at : {start}")
    elif list[start] == len(list):
        print("ID not found")
    else:
        return search(list,ID,start+1)


search(list,ID,start)
```

Output:

```
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\
Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_3.py
Enter number of inputs:3
Enter the ID:1
Enter the ID:2
Enter the ID:3
Enter ID to seach:2
ID found at : 1
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\
Python314\python.exe c:/Users/jaira/OneDrive/Documents/Desktop/SEM-2/DSA/DSA25106/CSE25106_DSA/lab1_3.py
Enter number of inputs:3
Enter the ID:1
Enter the ID:2
Enter the ID:3
Enter ID to seach:4
ID not found
PS C:\Users\jaira\OneDrive\Documents\Desktop\SEM-2\DSA\DSA25106\CSE25106_DSA>
```

Error Table:

Error	Rectification
Value Error: invalid literal for int() with base 10: '2 3'	Should take a int value only

Description:

- The program stores employee IDs in a list.
- The user enters the number of employee IDs.
- Each employee ID is added to the list.
- The user enters the ID that needs to be searched.
- The search() function checks the elements recursively.
- If the ID matches, its position is displayed.
- If the ID is not found, the program displays ID not found.

1d)

Aim:

- In a logistic department, the no of possible ways to arrange n diff parcels are calculated using the factorial formula(n!). Write a recursive formula to find the factorial of a given number

Program:

```python
#Factorial #CSE25106
def factorial(n):
    if n > 0:
        if n == 1:
            return 1
        else:
            return n*factorial(n-1)
    else:
```

```
        return "Please enter a positive number"
n = int(input("Enter the number:"))
print(factorial(n))
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Us
ers/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_4.py
Enter the number:5
120
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Us
ers/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_4.py
Enter the number:-1
Please enter a positive number
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>
```

Error Table:

Error	Rectification
Didn't implement for the negative values	Added else to make user to give positive input.

Description:

- The program calculates the factorial of a given number.
- Factorial is represented as n!.
- The factorial() function calls itself with n-1.
- If n is 1, the function returns 1.
- Otherwise, the current value of n is multiplied by the factorial of n-1.
- Negative or invalid values are handled by displaying a suitable message.

1e)

Aim:

- Print first n terms in a fibonacci number

Program:

```
#Fibonacci series #CSE25106
def fib(n):
    if n == 1:
        return 0
    if n == 2:
        return 1
    else:
        return fib(n-1) + fib(n-2)
def print_fib(n):
    for i in range(1, n+1):
        print(fib(i), end=" ")

n = int(input("Enter the number:"))
```

```
print_fib(n)
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Us
ers/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_5.py
Enter the number:5
0 1 1 2 3
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> & C:\Users\jaira\AppData\Local\Programs\Python\Python314\python.exe c:/Us
ers/jaira/OneDrive/Desktop/SEM-3/DSA/DSA25106/CSE25106_DSA/lab1_5.py
Enter the number:-3
Please enter a positive number
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>
```

Error Table:

Error	Rectification
Numbers are not printed if there are in the same function.	Added a new function with for loop for printing the numbers

Description:

- The program prints the first n terms of the Fibonacci series.
- The fib() function calculates Fibonacci numbers recursively.
- The first term is 0.
- The second term is 1.
- Each subsequent term is calculated by adding the previous two terms.
- The print_fib() function is used to print the Fibonacci terms.

Lab – 2

2a)

Aim:

- Write a program for the linear search algorithm

Program:

```
# Linear search #CSE25106
arr=[]
n = int(input("Enter no of elements:"))
while n < 1:
    print("no of elements should be greater than 0")
    n = int(input("Enter no of elements:"))
for i in range(n):
    elements = int(input("Enter the elements:"))
    arr.append(elements)
key = int(input("Enter element to search:"))
def linear_search(arr,key):
    for i in range(len(arr)):
        if arr[i] == key:
            return i
        else:
            i+= 1
    return -1
result = linear_search(arr,key)
if result != -1 :
    print("Element found at index ",result)
else:
    print("Element not found")
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA
25106\CSE25106_DSA\lab2_1.py"
Enter the elements:5
Enter element to search:3
Element found at index  1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_D
SA\lab2_1.py"
Enter no of elements:3
Enter the elements:1
Enter the elements:2
Enter the elements:3
Enter element to search:4
Element not found
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>
```

Error Table:

Error	Rectification
Forgot to return at the end which lead to the program returning none when the element is not found	Returned -1 at the very end which fixed the element not found case

Description:

- The program implements the Linear Search algorithm.
- The user enters the number of elements.
- The elements are stored in an array/list.
- The user enters the element to be searched.
- The linear_search() function checks the elements one by one.
- If the element matches the key, its index is returned.
- If the element is not found, -1 is returned.

2b)

Aim:

Write a program for binary search algorithm considering the elements to be sorted

Program:

```
# Binary search of sorted array #CSE25106
arr=[]
n = int(input("Enter no of elements:"))
for i in range(n):
    elements = int(input("Enter the elements in sorted order:"))
    arr.append(elements)
key = int(input("Enter element to search:"))
def binary_search(arr,key):
    low = 0
    high = len(arr) - 1
    while low < high:
        mid = (low+high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

result = binary_search(arr,key)
```

```
if result != -1 :
    print("Element found at index :",result)
else:
    print("Element not found")
```

Output:



Error Table:

Error	Rectification
Wrote the mid formula outside the while loop which gave the infinite loop error	Added the mid formula inside the while loop

Description:

- The program implements Binary Search on a sorted array.
- The user enters elements in sorted order.
- The algorithm uses low, high, and mid values.
- The middle element is calculated using (low + high) // 2.
- If the middle element equals the search key, its index is returned.
- If the key is greater than the middle element, the right half is searched.
- If the key is smaller, the left half is searched.
- If the element is not found, -1 is returned.

2c)

Aim:

Write a program for implementation of binary search along with the sorting the elements

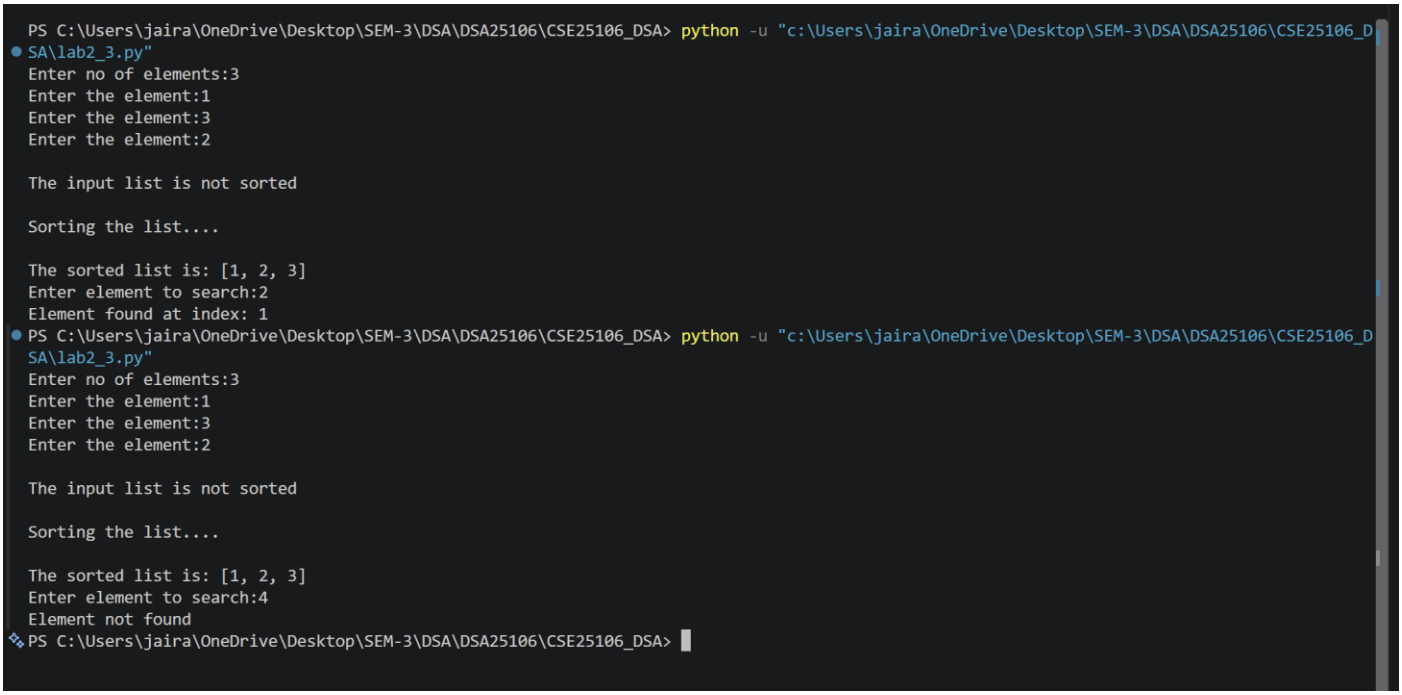
Program:

```
#Binary search of unsorted array. #CSE25106
arr = []
n = int(input("Enter no of elements:"))
```

```
for i in range(n):
    element = int(input("Enter the element:"))
    arr.append(element)
# To check and sort the list.
if arr == sorted(arr):
    print("\n The input list is already sorted")
else:
    print("\nThe input list is not sorted")
    print("\nSorting the list....")
    arr.sort()
print("\nThe sorted list is:",arr)
key = int(input("Enter element to search:"))
def binary_search(arr,key):
    low = 0
    high = len(arr) - 1
    while low < high:
        mid = (low+high)//2
        if arr[mid] == key:
            return mid
        elif arr[mid] < key:
            low = mid + 1
        else:
            high = mid - 1
    return -1

result = binary_search(arr,key)
if result != -1 :
    print("Element found at index:",result)
else:
    print("Element not found")
```

Output:



Error Table:

Error	Rectification
Used sorted method instead of which need to be referred to another variable as it returns a sorted array	Used the arr.sort() method which directly sorts the arr without using another variable

Description:

- The program performs Binary Search on user-provided elements.
- The input elements may initially be unsorted.
- The program checks whether the list is already sorted.
- If the list is not sorted, arr.sort() is used.
- The sorted list is displayed.
- The user enters the element to be searched.
- Binary Search is then performed on the sorted list.
- The middle element is compared with the search key.
- If the element is found, its index is displayed.
- Otherwise, Element not found is displayed.

Lab – 3

3a)

Aim:

- Write a python program to implement bubble sort where the input elements are taken from the user.

Program:

```python
# BUBBLE SORT # CSE25106
def main():
    arr = []
    n = int(input("Enter number of elements: "))
    for i in range(n):
        num = int(input(f"Enter the {i+1} element: "))
        arr.append(num)
    bubble_sort(arr)
    print("Sorted array:", arr)

def bubble_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        for j in range(n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

if __name__ == '__main__':
    main()
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA\lab3_1.py"
Enter number of elements: 4
Enter the 1 element: 1
Enter the 2 element: 4
Enter the 3 element: 6
Enter the 4 element: 2
Sorted array: [1, 2, 4, 6]
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA>
```

Error Table:

Error	Rectification
Gave the numbers in descending order	Changed the sign symbol while comparing the adjacent elements.

Description:

- The program implements the Bubble Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- Adjacent elements are compared with each other.
- If the first element is greater than the second element, they are swapped.
- This process is repeated for multiple passes.
- Larger elements gradually move toward the end of the list.
- The bubble_sort() function performs the sorting.

3b)

Aim:

➢ Write a python program to implement insertion sort where the input elements are taken from the user.

Program:

```
# INSERTION SORT # CSE25106
def main():
    arr = []
    n = int(input("Enter number of elements: "))
    for i in range(n):
        num = int(input(f"Enter the {i+1} element: "))
        arr.append(num)
    insertion_sort(arr)
    print(arr)

def insertion_sort(arr):
    for i in range(1, len(arr)):
        key = arr[i]
        j = i - 1
        while j >=0 and arr[j] > key:
            arr[j+1] = arr[j]
            j = j-1
        arr[j+1] = key

if __name__ == '__main__':
    main()
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\CSE25106_DSA\lab3_2,py"
Enter number of elements: 3
Enter the 1 element: 1
Enter the 2 element: 4
Enter the 3 element: 2
[1, 2, 4]
```

Error Table:

Error	Rectification
If range is taken incorrect, invalid output comes.	Take correct range.

Description:

- The program implements the Insertion Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- The first element is considered as the initially sorted portion.
- Each subsequent element is stored in a variable called `key`.
- The `key` is compared with the elements before it.
- Elements greater than the `key` are shifted one position to the right.
- The `key` is inserted into its correct position.
- This process continues until all elements are sorted.

3c)

Aim:

➢ Write a python program to implement selection sort where the input elements are taken from the user.

Program:

```
# SELECTION SORT # CSE25106
arr = []
n = int(input("Enter number of elements: "))
for i in range(n):
    num = int(input(f"Enter the {i+1} element: "))
    arr.append(num)
selection_sort(arr)
rint(arr)

def selection_sort(arr):
    for i in range(len(arr)-1):
        min_index = i
        for j in range(i+1, len(arr)):
            if arr[j] < arr[min_index]:
                min_index = j
        arr[i], arr[min_index] = arr[min_index], arr[i]
```

Output:



Error Table:

Error	Rectification
Wrong comparision.(>)	Use arr[j] < arr[min_index] and initialize min_index = i

Description:

- The program implements the Selection Sort algorithm.
- The user enters the number of elements.
- The elements are stored in a list.
- The algorithm starts from the first unsorted element.
- The smallest element in the remaining list is identified.
- Its index is stored in min_index.
- The smallest element is swapped with the current element.
- The process is repeated for all positions in the list.

Lab – 4

4a)

Aim:

- Write a python program to implement merge sort where the input elements are taken from the user.

Program:

```
# Merge SORT # CSE25106

def merge_sort(arr):
    if len(arr) <= 1:
        return arr

    mid = len(arr) // 2
    left = arr[:mid]
    right = arr[mid:]

    left = merge_sort(left)
    right = merge_sort(right)

    i = 0
    j = 0
    k = 0

    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            arr[k] = left[i]
            i += 1
        else:
            arr[k] = right[j]
            j += 1
        k += 1

    while i < len(left):
        arr[k] = left[i]
        i += 1
        k += 1

    while j < len(right):
        arr[k] = right[j]
        j += 1
        k += 1

    return arr
```

```python
n = int(input("Enter size of array: "))

arr = []

for i in range(n):
    arr.append(int(input(f"Enter {i+1} element: ")))

merge_sort(arr)

print("Sorted array:")

for element in arr:
    print(element, end=" ")
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-4\lab4_1.py"
Enter size of array: 4
Enter 1 element: 1
Enter 2 element: 5
Enter 3 element: 3
Enter 4 element: 8
Sorted array:
1 3 5 8
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> -1
-1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

Error Table:

Error	Rectification
Type Error: object of type 'NoneType' has no len()	Taken return arr at end of merge_sort.

Description:

- The program implements the Merge Sort algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The merge_sort() function recursively divides the array into two halves.
- The array is divided until each sub-array contains one or zero elements.
- The left and right portions are then sorted recursively.
- The sorted left and right arrays are compared element by element.
- The smaller element is placed into the original array.
- Remaining elements from either portion are then copied into the array.
- Finally, the completely sorted array is displayed.
- Merge Sort follows the divide-and-conquer approach.

4b)

Aim:

- Write a python program to implement quick sort where the input elements are taken from the user.

Program:

```
# Quick SORT # CSE25106

def quick_sort(a, low, high):
    if low < high:
        i = low
        j = high
        pivot = low

        while i < j:
            while i < len(a) and a[i] < a[pivot]:
                i += 1
            while a[j] > a[pivot]:
                j -= 1
            if i < j:
                a[i], a[j] = a[j], a[i]
        a[j], a[pivot] = a[pivot], a[j]
        quick_sort(a, low, j-1)
        quick_sort(a, j+1, high)

n = int(input("Enter size of array: "))
arr = []
for i in range(n):
    arr.append(int(input(f"enter {i+1} element: ")))
quick_sort(arr, 0, n-1)
print(arr)
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-4\lab4_2.py"
Enter size of array: 4
enter 1 element: 1
enter 2 element: 7
enter 3 element: 3
ent
[1,  Focus folder in explorer (ctrl + click)
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> -1
-1
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

Error Table:

Error	Rectification
i < len(a) checks the complete array instead of current subarray	Use i <= high and maintain correct partition boundaries

Description:

- The program implements the Quick Sort algorithm.
- The user enters the size of the array.
- The elements are taken from the user and stored in a list.
- The quick_sort() function is used to sort the array recursively.
- The first element of the current portion is selected as the pivot.
- Two pointers, i and j, are used to compare elements with the pivot.
- i moves from the left side and searches for an element greater than or equal to the pivot.
- j moves from the right side and searches for an element smaller than or equal to the pivot.
- If i < j, the two elements are swapped.
- After the partitioning process, the pivot is placed in its correct position.
- The array is then divided into left and right portions.
- Quick Sort is recursively applied to both portions.
- Finally, the sorted array is printed.

Week-5

5a)

Aim:

- Write a Python program to implement a Singly Linked List (SLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
# SinglyLinkedList # CSE25106
print("Implementation of SLL")

class Node:
    def __init__(self,data):
        self.data = data
        self.next = None

class SingleLinkedList:
    def __init__(self):
        self.head = None
    #Creation
    def create(self):
        n = int(input("Enter no of elements:"))
        for i in range(n):
            data = int(input("Enter the values:"))
            new = Node(data)
            if self.head is None:
```

```
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
    #Insert begin
    def insert_begin(self,data):
        new = Node(data)
        new.next = self.head
        self.head = new
    #Insert end
    def insert_end(self,data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
    #Insert at specific index
    def insert_index(self,index,data):
        if index == 0:
            self.insert_begin(data)
            return
        elif(index < 0):
            print("Invalid index")
            return
        else:
            new = Node(data)
            temp = self.head
            for i in range(index-1):
                if temp.next is None:
                    print("Invalid index")
                    return
                temp = temp.next
            new.next = temp.next
            temp.next = new
    #Delete specific value
    def delete(self,data):
        if self.head is None:
            print("No data")
        else:
            temp = self.head
            if temp and temp.data == data:
                self.head = temp.next
                print("Value deleted")
                return
```

```
            while temp.next and temp.next.data != data:
                temp = temp.next
            if temp.next is None:
                print("Value not present")
            else:
                temp.next = temp.next.next
                print("Value deleted")
    #Delete begin
    def delete_begin(self):
        if self.head is None:
            print("No data to delete")
        else:
            temp = self.head
            self.head = temp.next
            print("Deleted value = ",temp.data)
    #Delete end
    def delete_end(self):
        if self.head is None:
            print("No data to delete")
        elif self.head.next is None:
            self.head = None
        else:
            temp = self.head
            temp1 = temp
            while temp.next:
                temp1 = temp
                temp = temp.next
            temp1.next = None
    #Count
    def count(self):
        if self.head is None:
            print("No data to count")
        else:
            count = 0
            temp = self.head
            while temp:
                count += 1
                temp = temp.next
            print(f"Number of nodes = {count}")
    #Display
    def display(self):
        if self.head is None:
            print("No data")
        else:
            temp = self.head
            while temp:
                print(temp.data,end="-->")
                temp = temp.next
            print("None")
```

```
#Calling main function
SLL = SingleLinkedList()

while True:
    print("1.Create SLL")
    print("2.Insert at beginning")
    print("3.Insert at ending")
    print("4.Insert at index")
    print("5.Delete by value")
    print("6.Delete first node")
    print("7.Delete last node")
    print("8.Count no of nodes")
    print("9.Display / Traverse")
    print("10.Exit")

    choice = int(input("Enter your choice:"))

    if(choice == 1):
        SLL.create()
        print("SLL is created")
    elif(choice == 2):
        data = int(input("Enter data to insert: "))
        SLL.insert_begin(data)
        print("Data is inserted at beginning")
    elif(choice == 3):
        data = int(input("Enter data to insert: "))
        SLL.insert_end(data)
        print("Data is inserted at ending")
    elif(choice == 4):
        data = int(input("Enter data to insert: "))
        index = int(input("Enter the index:"))
        SLL.insert_index(index,data)
        print("Data is inserted at specific index")
    elif(choice == 5):
        data = int(input("Enter data to insert: "))
        SLL.delete(data)
    elif(choice == 6):
        SLL.delete_begin()
    elif(choice == 7):
        SLL.delete_end()
    elif(choice == 8):
        SLL.count()
    elif(choice == 9):
        SLL.display()
    elif(choice == 10):
        print("Exiting the program")
        break
    else:
```

```
        print("Invalid choice")
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-5\lab5_1.py"
Implementation of SLL
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:1
Enter no of elements:2
Enter the values:1
Enter the values:2
SLL is created
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:2
Enter data to insert: 0
Data is inserted at beginning
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:3
Enter data to insert: 3
Data is inserted at ending
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:9
0-->1-->2-->3-->None
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:8
Number of nodes = 4
1.Create SLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice:10
Exiting the program
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

Error Table:

Error	Rectification
The program may display an incorrect message for insertion at a specific index even when the index is invalid	Check whether the index is valid before displaying the successful insertion message.

Description:

- The program implements a Singly Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains two parts: data and next.
- data stores the value of the node.
- next stores the reference to the next node.
- The Single Linked List class manages the linked list.
- The head variable stores the reference to the first node.
- It provides a function to count the number of nodes.
- It provides a function to display all the nodes.
- The program continues running until the user selects the Exit option.

5b)

Aim:

- Write a Python program to implement a Double Linked List (DLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```python
# DoubleLinkedList # CSE25106

print("Implementation of DLL")

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoubleLinkedList:
    def __init__(self):
        self.head = None
    # Creation
    def create(self):
        n = int(input("Enter no of elements: "))
        for i in range(n):
            data = int(input("Enter the values: "))
            new = Node(data)
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new
                new.prev = temp

    # Insert begin
    def insert_begin(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            new.next = self.head
            self.head.prev = new
            self.head = new

    # Insert end
    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new
```

```
            new.prev = temp

    # Insert at specific index
    def insert_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return
        if index == 0:
            self.insert_begin(data)
            return
        if self.head is None:
            print("Invalid index")
            return
        temp = self.head
        for i in range(index - 1):
            if temp.next is None:
                print("Invalid index")
                return
            temp = temp.next
        new = Node(data)
        new.next = temp.next
        new.prev = temp
        if temp.next is not None:
            temp.next.prev = new
        temp.next = new

    # Delete specific value
    def delete(self, data):
        if self.head is None:
            print("No data")
            return
        temp = self.head
        while temp and temp.data != data:
            temp = temp.next
        if temp is None:
            print("Value not present")
        else:
            if temp.prev is not None:
                temp.prev.next = temp.next
            else:
                self.head = temp.next
            if temp.next is not None:
                temp.next.prev = temp.prev
            print("Value deleted")

    # Delete begin
    def delete_begin(self):
        if self.head is None:
```

```
            print("No data to delete")
        else:
            temp = self.head
            self.head = temp.next
            if self.head is not None:
                self.head.prev = None
            print("Deleted value = ", temp.data)

    # Delete end
    def delete_end(self):
        if self.head is None:
            print("No data to delete")
        elif self.head.next is None:
            print("Deleted value = ", self.head.data)
            self.head = None

        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            print("Deleted value = ", temp.data)
            temp.prev.next = None

    # Count
    def count(self):
        if self.head is None:
            print("No data to count")
        else:
            count = 0
            temp = self.head
            while temp:
                count += 1
                temp = temp.next
            print(f"Number of nodes = {count}")

    # Display / Traverse
    def display(self):
        if self.head is None:
            print("No data")
        else:
            temp = self.head
            while temp:
                print(temp.data, end=" <--> ")
                temp = temp.next

            print("None")
```

```
DLL = DoubleLinkedList()

while True:
    print("\n1.Create DLL")
    print("2.Insert at beginning")
    print("3.Insert at ending")
    print("4.Insert at index")
    print("5.Delete by value")
    print("6.Delete first node")
    print("7.Delete last node")
    print("8.Count no of nodes")
    print("9.Display / Traverse")
    print("10.Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        DLL.create()
        print("DLL is created")

    elif choice == 2:
        data = int(input("Enter data to insert: "))
        DLL.insert_begin(data)
        print("Data is inserted at beginning")

    elif choice == 3:
        data = int(input("Enter data to insert: "))
        DLL.insert_end(data)
        print("Data is inserted at ending")

    elif choice == 4:
        data = int(input("Enter data to insert: "))
        index = int(input("Enter the index: "))
        DLL.insert_index(index, data)

    elif choice == 5:
        data = int(input("Enter data to delete: "))
        DLL.delete(data)

    elif choice == 6:
        DLL.delete_begin()

    elif choice == 7:
        DLL.delete_end()

    elif choice == 8:
        DLL.count()
```

```
    elif choice == 9:
        DLL.display()

    elif choice == 10:
        print("Exiting the program")
        break

    else:
        print("Invalid choice")
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-5\lab5_2.py"
Implementation of DLL

1.Create DLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 1
Enter no of elements: 2
Enter the values: 1
Enter the values: 2
DLL is created

1.Create DLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 2
Enter data to insert: 0
Data is inserted at beginning
```

Error Table:

Error	Rectification
The prev and next links may not be updated correctly during insertion or deletion.	Update both links whenever a node is inserted or deleted.

Description:

- The program implements a Doubly Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains three parts: data, prev, and next.
- data stores the value of the node.
- prev stores the reference to the previous node.

- next stores the reference to the next node.
- The Double Linked List class is used to manage the linked list.
- The `head` variable stores the reference to the first node.
- It provides a function to count the number of nodes.
- It provides a function to display/traverse the list.
- During insertion and deletion, both prev and next links are updated.
- The program continues until the user selects the Exit option.

5c)

Aim:

- Write a Python program to implement a Circular Linked List (CLL) with the following operations:

1. Create LL
2. Insert at Beginning
3. Insert at End
4. Insert at Index
5. Delete by Value
6. Delete Prior Node
7. Delete Last Node
8. Count Number of Nodes
9. Display/Traverse
10. Exit

Program:

```
# CircularLinkedList # CSE25106

print("Implementation of CLL")

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class CircularLinkedList:
    def __init__(self):
        self.head = None
```

```
    def create(self):
        n = int(input("Enter no of elements: "))

        for i in range(n):
            data = int(input("Enter the values: "))
            new = Node(data)

            if self.head is None:
                self.head = new
                new.next = self.head
            else:
                temp = self.head

                while temp.next != self.head:
                    temp = temp.next

                temp.next = new
                new.next = self.head

    def insert_begin(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            new.next = self.head
            self.head = new
            temp.next = self.head
    def insert_end(self, data):
        new = Node(data)

        if self.head is None:
            self.head = new
            new.next = self.head
        else:
            temp = self.head

            while temp.next != self.head:
                temp = temp.next
```

```
            temp.next = new
            new.next = self.head

    def insert_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        if self.head is None:
            print("Invalid index")
            return

        temp = self.head

        for i in range(index - 1):
            if temp.next == self.head:
                print("Invalid index")
                return

            temp = temp.next

        new = Node(data)
        new.next = temp.next
        temp.next = new

    def delete(self, data):
        if self.head is None:
            print("No data")
            return

        if self.head.data == data:

            if self.head.next == self.head:
                self.head = None
                print("Value deleted")
                return

            temp = self.head

            while temp.next != self.head:
                temp = temp.next

            temp.next = self.head.next
```

```
            self.head = self.head.next

            print("Value deleted")
            return

        temp = self.head

        while temp.next != self.head and temp.next.data != data:
            temp = temp.next

        if temp.next == self.head:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")
    def delete_begin(self):
        if self.head is None:
            print("No data to delete")
            return
        if self.head.next == self.head:
            print("Deleted value = ", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next != self.head:
            temp = temp.next

        print("Deleted value = ", self.head.data)

        temp.next = self.head.next
        self.head = self.head.next
    def delete_end(self):
        if self.head is None:
            print("No data to delete")
            return
        if self.head.next == self.head:
            print("Deleted value = ", self.head.data)
            self.head = None
            return

        temp = self.head

        while temp.next.next != self.head:
            temp = temp.next
```

```
        print("Deleted value = ", temp.next.data)

        temp.next = self.head

    def count(self):
        if self.head is None:
            print("No data to count")
            return

        count = 0
        temp = self.head

        while True:
            count += 1
            temp = temp.next

            if temp == self.head:
                break

        print(f"Number of nodes = {count}")

    def display(self):
        if self.head is None:
            print("No data")
            return

        temp = self.head

        while True:
            print(temp.data, end="-->")
            temp = temp.next

            if temp == self.head:
                break

        print("HEAD")

CLL = CircularLinkedList()

while True:
    print("\n1.Create CLL")
    print("2.Insert at beginning")
    print("3.Insert at ending")
    print("4.Insert at index")
    print("5.Delete by value")
    print("6.Delete first node")
    print("7.Delete last node")
```

```
    print("8.Count no of nodes")
    print("9.Display / Traverse")
    print("10.Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        CLL.create()
        print("CLL is created")

    elif choice == 2:
        data = int(input("Enter data to insert: "))
        CLL.insert_begin(data)
        print("Data is inserted at beginning")

    elif choice == 3:
        data = int(input("Enter data to insert: "))
        CLL.insert_end(data)
        print("Data is inserted at ending")

    elif choice == 4:
        data = int(input("Enter data to insert: "))
        index = int(input("Enter the index: "))
        CLL.insert_index(index, data)

    elif choice == 5:
        data = int(input("Enter data to delete: "))
        CLL.delete(data)

    elif choice == 6:
        CLL.delete_begin()

    elif choice == 7:
        CLL.delete_end()

    elif choice == 8:
        CLL.count()

    elif choice == 9:
        CLL.display()

    elif choice == 10:
        print("Exiting the program")
        break

    else:
        print("Invalid choice")
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-5\lab5_3.py"
Implementation of CLL

1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 1
Enter no of elements: 2
Enter the values: 1
Enter the values: 2
CLL is created

1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 9
1-->2-->HEAD

1.Create CLL
2.Insert at beginning
3.Insert at ending
4.Insert at index
5.Delete by value
6.Delete first node
7.Delete last node
8.Count no of nodes
9.Display / Traverse
10.Exit
Enter your choice: 10
Exiting the program
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

Error Table:

Error	Rectification
No major error is present in the given program.	No major rectification is required.

Description:

- The program implements a Circular Linked List using Python classes.
- The Node class is used to create individual nodes.
- Each node contains two parts: data and next.
- `data` stores the value of the node.
- `next` stores the reference to the next node.
- In a Circular Linked List, the last node points back to the first node.
- The Circular Linked List class is used to manage the linked list.
- The `head` variable stores the reference to the first node.
- The program provides an option to create a circular linked list.

- It provides a function to display/traverse the circular linked list.
- During traversal, the program stops when it reaches the `head` again.
- The program handles the special case where the list contains only one node.
- A menu-driven approach is used to perform different CLL operations.
- The program continues until the user selects the Exit option.

Week-6

6a)

Aim:

- Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Array.

Program:

```
# Stack using Array # CSE25106

class Stack:
    def __init__(self,size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

    def push(self,item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top += 1
            self.stack[self.top] = item
            print(item,"pushed into the stack")
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top -= 1
            print(item,"popped from the stack")
    def peek(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("Top element:",self.stack[self.top])
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack:")
            for i in range(self.top,-1,-1):
                print(self.stack[i])

size = int(input("Enter the size of the stack:"))
s = Stack(size)
```

```
while True:
    print("--STACK MENU--")
    print("1.Push")
    print("2.Pop")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")

    choice = int(input("Enter your choice:"))

    if choice == 1:
        item = int(input("Enter the element to push:"))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-6\lab6_1.py"
Enter the size of the stack:5
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to push:1
1 pushed into the stack
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:1
Enter the element to push:2
2 pushed into the stack
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:2
2 popped from the stack
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:3
Top element: 1
```

```
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:4
The elements of the stack:
1
--STACK MENU--
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice:5
Exiting program
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

Error Table:

Error	Rectification
In peek, self.peek is taken in 'if' does not give correct answer for stack underflow.	Self.top is taken in 'if' to give correct output.

Description:

- The Stack class implements a stack using a Python list.
- size stores the maximum size of the stack.
- stack stores the stack elements.
- top keeps track of the top position. Initially, top = -1, meaning the stack is empty.
- Push adds an element to the top of the stack.
- Pop removes the top element from the stack.
- Peek displays the top element without removing it.
- Display prints all elements from top to bottom.
- Stack Overflow occurs when trying to push into a full stack.
- Stack Underflow occurs when trying to pop from an empty stack.
- The program uses a menu-driven approach with options for Push, Pop, Peek, Display, and Exit.

6b)

Aim:

- Write a Python program to implement stack operations such as Push, Pop, Peek, and Display using an Linked List.

Program:

```python
# Stack using Ll # CSE25106

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

```
class Stack:
    def __init__(self):
        self.top = None

    def push(self, item):
        new = Node(item)
        new.next = self.top
        self.top = new
        print(item, "pushed into the stack")

    def pop(self):
        if self.top is None:
            print("Stack Underflow")
        else:
            temp = self.top
            self.top = self.top.next
            print(temp.data, "popped from the stack")

    def peek(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("Top element:", self.top.data)

    def display(self):
        if self.top is None:
            print("Stack is empty")
        else:
            print("The elements of the stack:")
            temp = self.top
            while temp is not None:
                print(temp.data)
                temp = temp.next


s = Stack()

while True:
    print("\n--STACK MENU--")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display")
    print("5. Exit")
```

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)
    elif choice == 2:
        s.pop()
    elif choice == 3:
        s.peek()
    elif choice == 4:
        s.display()
    elif choice == 5:
        print("Exiting program")
        break
    else:
        print("Invalid choice")
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-6\lab6_2.py"

--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 1
1 pushed into the stack

--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 1
Enter the element to push: 2
2 pushed into the stack

--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the stack:
2
1

--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 2
2 popped from the stack
```

```
--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 4
The elements of the stack:
1

--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 3
Top element: 1

--STACK MENU--
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice: 5
Exiting program
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

Error Table:

Error	Rectification
No major error in the linked-list implementation.	Not required.

Description:

- The stack is implemented using a singly linked list.
- Each node contains data and a next pointer.
- top points to the top node of the stack.
- Push inserts a new node at the beginning of the linked list.
- Pop removes the node pointed to by top.
- Peek displays the data of the top node without removing it.
- Display traverses the linked list from top to the last node.
- If top is None, the stack is empty.
- Stack Overflow is not a fixed-size issue here because the stack grows dynamically, subject to available memory.

Week-7

7a)

Aim:

- Write a Python program to implement queue operations such as Push, Pop, Peek, and Display using an array.

Program:

```
class CircularQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None]*size
        self.front = -1
        self.rear = -1
    def enqueue(self, item):
        if (self.rear + 1) % self.size == self.front:
            print("queue over flow")
            return
        if self.front == -1:
            self.front = self.rear = 0
            self.queue[self.rear] = item
            print(f"{item} inserted")
            return
        self.rear = (self.rear+1) % self.size
        self.queue[self.rear] = item
        print(f"{item} inserted")
        return
    def dequeue(self):
        if self.front == -1:
            print("queue under flow")
            return
        item = self.queue[self.front]
        self.front = (self.front+1) % self.size
        if self.front == (self.rear+1) % self.size:
            self.front = self.rear = -1
        print(f"{item} deleted")
        return
    def peek(self):
        if self.front == -1:
            print("the queue is empty")
            return
        item = self.queue[self.front]
        print(f"{item} is the font element")
        return
    def display(self):
        if self.front == -1:
            print("the queue is empty")
```

```
            return
        i = self.front
        while i != (self.rear + 1)%self.size:
            print(self.queue[i], end="|")
            i = (i+1) % self.size
        print("")
        return

def main():
    print("CSE25106, Ch. Jai ram charan.")
    queue = CircularQueue(5)
    while True:
        print("1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
        choice = int(input("Enter Your Choice: "))
        if choice == 1:
            queue.enqueue(int(input("Enter the item: ")))
        elif choice == 2:
            queue.dequeue()
        elif choice == 3:
            queue.peek()
        elif choice == 4:
            queue.display()
        elif choice == 5:
            print("Exiting the program.....")
            break
        else:
            print("Invalid choice")

if __name__ == '__main__':
    main()
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-7\circular_queue.py"
CSE25106, Ch. Jai ram charan.
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 1
Enter the item: 4
4 inserted
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 2
4 deleted
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 3
the queue is empty
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 4
the queue is empty
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 5
Exiting the program.....
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

Error Table:

Error	Rectification
Unused Space Error: After several dequeues, empty positions at the beginning cannot be reused in a linear array queue.	Use a circular queue when reuse of empty positions is required.

Description:

- The queue is implemented using a fixed-size array.
- Each element is stored in a separate array position.
- front points to the first element of the queue.
- rear points to the last element of the queue.
- Enqueue inserts a new element at the rear of the queue.
- Dequeue removes the element from the front of the queue.
- Peek displays the element at the front without removing it.
- Display traverses the queue from front to rear.

• If the queue is empty, front and rear indicate the empty condition.
• Queue Overflow occurs when an element is inserted into a full array queue and Queue Underflow occurs when an element is deleted from an empty queue.

7b)

Aim:

- Write a Python program to implement queue operations such as Push, Pop, Peek, and Display using a Linked List.

Program:

```
class Queue:
    def __init__(self, size):
        self.size = size
        self.queue = [None]*size
        self.front = -1
        self.rear = -1
    def enqueue(self, item):
        if self.rear == self.size - 1:
            print("Queue overflow")
            return
        if self.front == -1 and self.rear == -1:
            self.front = self.rear = 0
            self.queue[self.rear] = item
            print(f"{item} is inserted into queue")
            return
        self.rear += 1
        self.queue[self.rear] = item
        print(f"{item} is inserted")
        return
    def dequeue(self):
        if self.front == -1:
            print("queue over flow")
            return
        item = self.queue[self.front]
        self.front += 1
        if self.front > self.rear:
            self.front = self.rear = -1
        print(f"{item} deleted from the queue")
        return
    def peek(self):
        if self.front == -1:
            print("the queue is empty")
            return
        item = self.queue[self.front]
        print(f"the front element is {item}")
        return
    def display(self):
```

```
        if self.front == -1:
            print("th4e queue is empty")
            return
        for i in range(self.front, self.rear+1):
            print(f"{self.queue[i]}", end=" | ")
        print("")
        return

def main():
    print("CSE25106, Ch. Jai ram charan.")
    queue = Queue(5)
    while True:
        print("1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
        choice = int(input("Enter Your Choice: "))
        if choice == 1:
            queue.enqueue(int(input("Enter the item: ")))
        elif choice == 2:
            queue.dequeue()
        elif choice == 3:
            queue.peek()
        elif choice == 4:
            queue.display()
        elif choice == 5:
            print("Exiting the program")
            break
        else:
            print("Invalid choice")

if __name__ == '__main__':
    main()
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-7\queue_array.py"
CSE25106, Ch. Jai ram charan.
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 1
Enter the item: 2
2 is inserted into queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 1
Enter the item: 3
3 is inserted
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 2
2 deleted from the queue
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 3
the front element is 3
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 4
3 |
1.Enqueue
2.Dequeue
3.Peek
```

Error Table:

Error	Rectification
Incorrectly changing the next pointer can break the linked list.	Maintain the correct next links during insertion and deletion

Description:

- The queue is implemented using a singly linked list.
- Each node contains data and a next pointer.
- front points to the first node of the queue.
- rear points to the last node of the queue.
- Enqueue creates a new node and inserts it at the rear.
- Dequeue removes the node pointed to by front.
- Peek displays the data of the front node without removing it.
- Display traverses the linked list from front to rear.
- If front is None, the queue is empty.
- Queue Overflow is not a fixed-size issue because the queue grows dynamically, subject to available memory.
- Queue Underflow occurs when dequeue is attempted on an empty queue.

7c)

Aim:

- Write a Python program to implement queue operations such as Push, Pop, Peek, and Display using an Circular Linked List.

Program:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
class Queue:
    def __init__(self):
        self.front = None
        self.rear = None
    def enqueue(self, item):
        node = Node(item)
        if self.front is None:
            self.rear = node
            self.front = node
            print(f"{item} inserted")
            return
        self.rear.next = node
        self.rear = node
        print(f"{item} is inserted")
        return
    def dequeue(self):
        if self.front is None:
            print("queue under flow")
            return
        item = self.front.data
        self.front = self.front.next
        print(f"{item} is deleted")
        return
    def peek(self):
        if self.front is None:
            print("queue is empty")
            return
        item = self.front.data
        print(f"{item} is the front item")
        return
    def display(self):
        if self.front is None:
            print("queue is Empty")
            return
        temp = self.front
        while temp:
            print(temp.data, end=" | ")
```

```
            temp = temp.next
        print("")
        return
def main():
    print("CSE25106, Ch . Jai ram charan.")
    queue = Queue()
    while True:
        print("1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
        choice = int(input("Enter Your Choice: "))
        if choice == 1:
            queue.enqueue(int(input("Enter the item: ")))
        elif choice == 2:
            queue.dequeue()
        elif choice == 3:
            queue.peek()
        elif choice == 4:
            queue.display()
        elif choice == 5:
            print("Exiting the program.....")
            break
        else:
            print("Invalid choice")

if __name__ == '__main__':
    main()
```

Output:

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-7\queue_linked_list.py"
CSE25106, Ch . Jai ram charan.
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 1
Enter the item: 2
2 inserted
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 1
Enter the item: 4
4 is inserted
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 2
2 is deleted
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 3
4 is the front item
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter Your Choice: 4
4
1.Enqueue
2.Dequeue
```

Error Table:

Error	Rectification
Display may stop at the physical end of the array instead of continuing from the beginning.	Traverse using circular indexing until the rear position is reached.

Description:

- The circular queue is implemented using a fixed-size array.
- The last position of the array is connected logically to the first position.
- front points to the first element of the queue.
- rear points to the last element of the queue.
- Enqueue inserts a new element at the rear.
- Dequeue removes the element from the front.
- After reaching the last array position, rear wraps around to the first position.
- Similarly, front wraps around when it reaches the last position.
- Peek displays the front element without removing it.
- Display traverses the queue from front to rear while following the circular order.
- The queue is empty when the defined empty condition for front and rear is satisfied.
- Queue Overflow occurs when all positions in the circular queue are occupied.

Week-8

8a)

Aim:

- Write a Python program to implement Binary Tree with its operations using array.

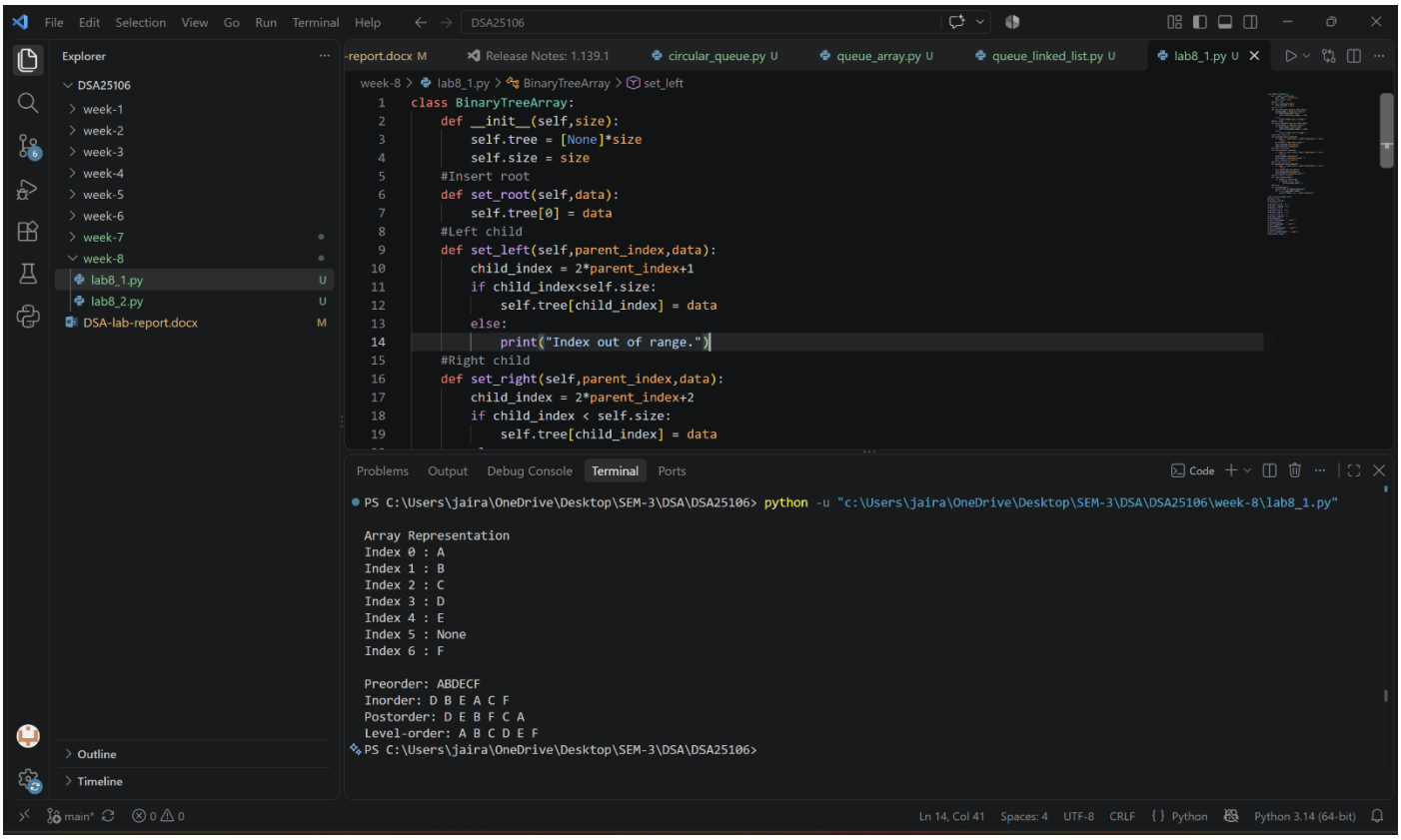
Program:

```
class BinaryTreeArray:
    def __init__(self,size):
        self.tree = [None]*size
        self.size = size
    def set_root(self,data):
        self.tree[0] = data
    def set_left(self,parent_index,data):
        child_index = 2*parent_index+1
        if child_index<self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range.")
    def set_right(self,parent_index,data):
        child_index = 2*parent_index+2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range.")
    def preorder(self,index=0):
        if index >= self.size or self.tree[index] is None:
            return
        print(self.tree[index],end="")
        self.preorder(2*index+1)
        self.preorder(2*index+2)
    def inorder(self,index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.inorder(2*index+1)
        print(self.tree[index],end=" ")
        self.inorder(2*index+2)
    def postorder(self,index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.postorder(2*index+1)
        self.postorder(2*index+2)
        print(self.tree[index],end=" ")
    def level_order(self):
        for value in self.tree:
            if value is not None:
```

```
                print(value,end=" ")
    def display(self):
        print("\nArray Representation")
        for i in range(self.size):
            print(f"Index {i} : {self.tree[i]}")

tree = BinaryTreeArray(7)
#Create tree
tree.set_root('A')
# Children of A
tree.set_left(0, 'B')
tree.set_right(0, 'C')
# Children of B
tree.set_left(1, 'D')
tree.set_right(1, 'E')
# Right child of C
tree.set_right(2, 'F')
tree.display()
print("\nPreorder: ", end="")
tree.preorder()
print("\nInorder: ", end="")
tree.inorder()
print("\nPostorder: ", end="")
tree.postorder()
print("\nLevel-order: ", end="")
tree.level_order()
```

Output:



Error Table:

Error	Rectification
Accessing an index outside the array size can cause Index Error.	Check the array boundaries before accessing an index.

Description:

- The binary tree is represented using an array, where each position stores a node of the tree.
- The first element of the array represents the root node.
- For a node at index i, its left child is stored at 2*i + 1.
- For a node at index i, its right child is stored at 2*i + 2.
- The binary tree can be traversed using preorder, inorder, and postorder methods.
- New elements are inserted into the appropriate array positions while maintaining the binary-tree structure.
- Empty positions in the array indicate that the corresponding node does not exist.

8b)

Aim:

➢ Write a Python program to implement Binary Tree with its operations using Linked list.

Procedure:

```
class Node:
    def __init__(self,data):
        self.left = None
        self.right = None
        self.data = data

class BinaryTree:
    def __init__(self):
        self.root = None
    def create_tree(self):
        self.root = Node('A')
        # Left and right children of A
        self.root.left = Node('B')
        self.root.right = Node('C')
        # Children of B
        self.root.left.left = Node('D')
        self.root.left.right = Node('E')
        # Right child of C
        self.root.right.right = Node('F')
    def preorder(self, node):
        if node is None:
            return
        print(node.data, end=" ")
```

```
        self.preorder(node.left)
    def inorder(self, node):
        if node is None:
            return
        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)
    def postorder(self, node):
        if node is None:
            return
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")
    def level_order(self):
        if self.root is None:
            return
        queue = [self.root]
        while queue:
            node = queue.pop(0) # Remove first node
            print(node.data, end=" ")
            # Add left child
            if node.left is not None:
                queue.append(node.left)
            # Add right child
            if node.right is not None:
                queue.append(node.right)

tree = BinaryTree()
tree.create_tree()

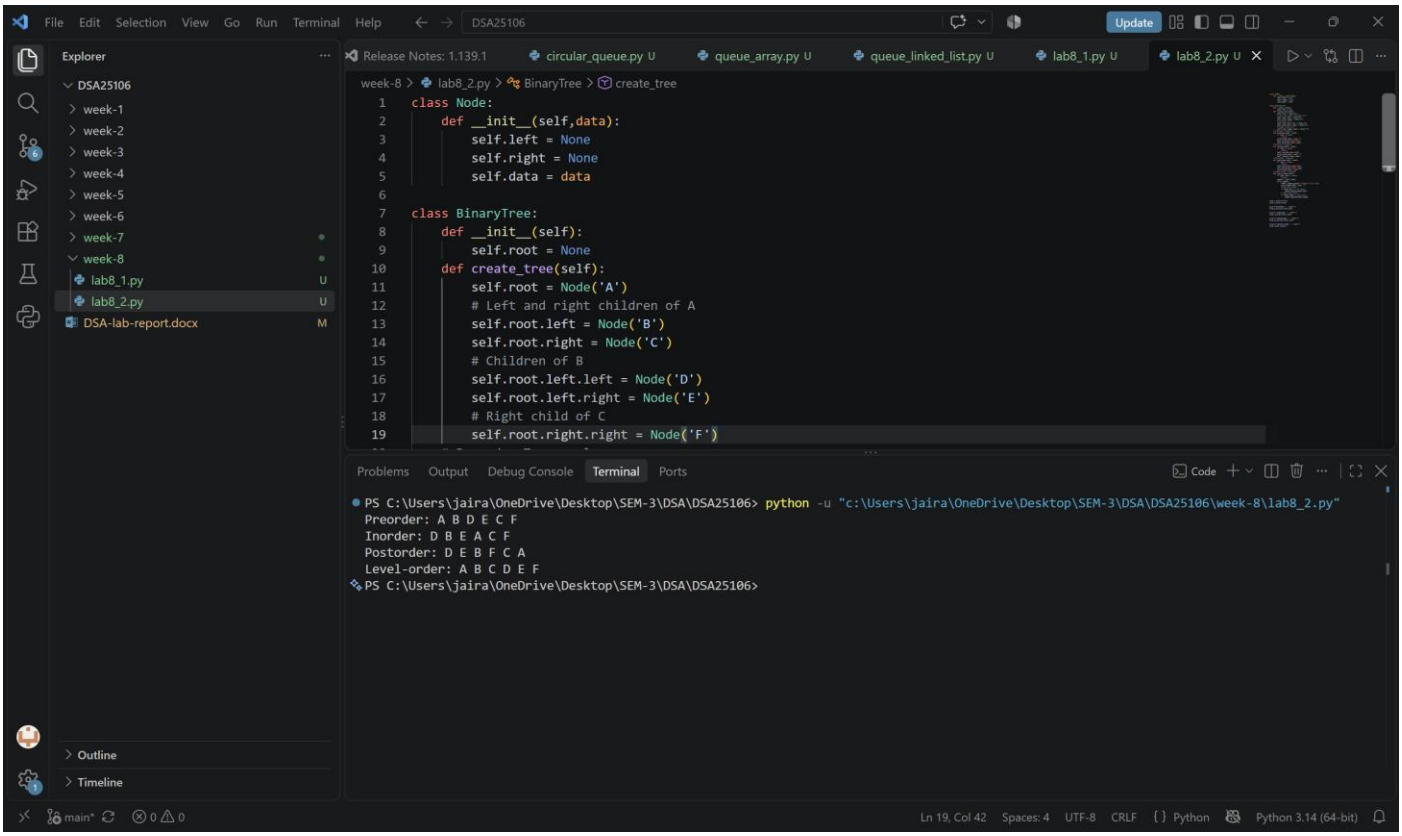
print("Preorder: ", end="")
tree.preorder(tree.root)

print("\nInorder: ", end="")
tree.inorder(tree.root)

print("\nPostorder: ", end="")
tree.postorder(tree.root)

print("\nLevel-order: ", end="")
tree.level_order()
```

Output:



Error Table:

Error	Rectification
Accessing a None node may cause an Attribute Error.	Check whether the node is None before accessing its data or pointers.

Description:

- Each tree node contains data, left pointer, and right pointer.
- The root pointer stores the address of the first node of the binary tree.
- The left pointer stores the address of the left child.
- The right pointer stores the address of the right child.
- New nodes are dynamically created and connected to the appropriate parent node.
- Memory is allocated dynamically as new nodes are created.
- The tree can be traversed using preorder, inorder, and postorder methods.
- A node with both left and right pointers as None is called a leaf node.
- Linked-list representation allows the tree to grow dynamically without a fixed size.
- Each node is connected to its children using pointers, providing a flexible representation of a binary tree

Week-9

9)

Aim:

- Write a Python program to implement Binary Search Tree with its operations

Program:

```
# Binary Search Tree
#CSE25106, Ch. Jai Ram Charan.
# Binary Search Tree
#CSE25106, Ch. Jai Ram Charan.
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None


class BinarySearchTree:
    def insert(self, root, data):
        if root is None:
            return Node(data)
        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)
        return root
    def search(self, root, data):
        if root is None:
            return False
        if root.data == data:
            return True
        elif data < root.data:
            return self.search(root.left, data)
        else:
            return self.search(root.right, data)
    def find_min(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current
    def delete(self, root, data):
        if root is None:
            return root
        if data < root.data:
            root.left = self.delete(root.left, data)
        elif data > root.data:
            root.right = self.delete(root.right, data)
```

```
        else:
            # Case 1: No child
            if root.left is None and root.right is None:
                return None
            # Case 2: Only right child
            elif root.left is None:
                return root.right
            # Case 3: Only left child
            elif root.right is None:
                return root.left
            # Case 4: Two children
            else:
                temp = self.find_min(root.right)
                root.data = temp.data
                root.right = self.delete(root.right, temp.data)
        return root
    def inorder(self, root):
        if root is not None:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)
    def preorder(self, root):
        if root is not None:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)
    def postorder(self, root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

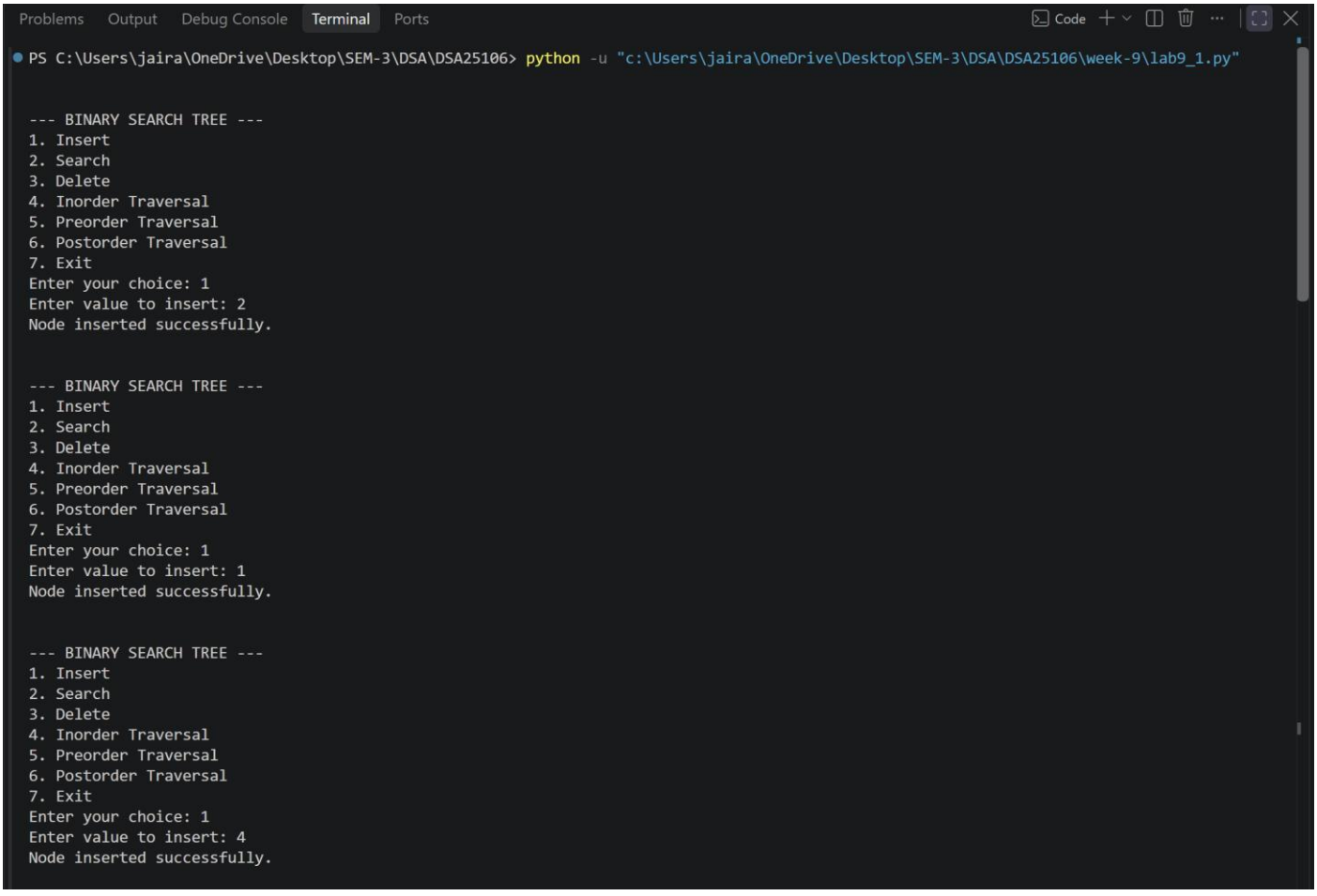
bst = BinarySearchTree()
root = None

while True:
    print("\n\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

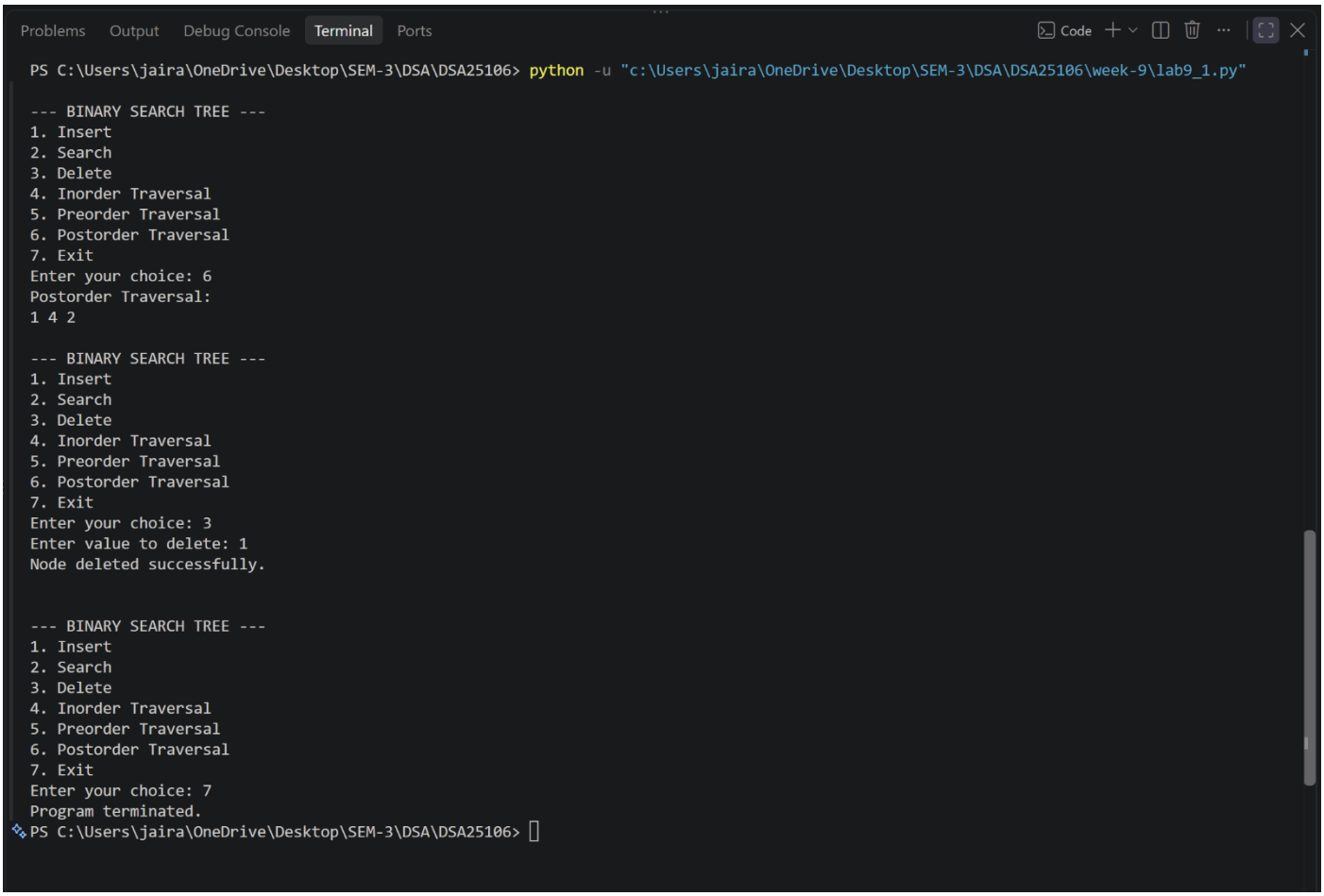
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter value to insert: "))
        root = bst.insert(root, data)
```

```
        print("Node inserted successfully.")
    elif choice == 2:
        data = int(input("Enter value to search: "))
        if bst.search(root, data):
            print("Element found.")
        else:
            print("Element not found.")
    elif choice == 3:
        data = int(input("Enter value to delete: "))
        root = bst.delete(root, data)
        print("Node deleted successfully.")
    elif choice == 4:
        print("Inorder Traversal:")
        bst.inorder(root)
    elif choice == 5:
        print("Preorder Traversal:")
        bst.preorder(root)
    elif choice == 6:
        print("Postorder Traversal:")
        bst.postorder(root)
    elif choice == 7:
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```



Output:



Error Table:

Error	Rectification
Accessing a None node may cause an Attribute Error.	Check whether the node is None before accessing its data or pointers.

Description:

- The program implements a Binary Search Tree (BST) using Python classes and objects.
- Each node stores a data value and references to its left and right child nodes.
- The insertion operation places each new value in its correct position according to BST rules.
- The search operation checks whether a specified value exists in the tree.
- The deletion operation removes a node and adjusts the tree structure based on the number of children.
- The minimum value is found by traversing the left child nodes until the leftmost node is reached.
- The traversal operations display the tree elements using inorder, preorder, and postorder methods.

Week-10

10)

Aim:

- Python program to implement AVL Tree with all its operations.

Program:

```
# AVL Tree
# CSE25106, Ch. Jai Ram Charan.
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None
        self.height = 1
class AVLTree:
    def height(self, root):
        if root is None:
            return 0
        return root.height
    def balance(self, root):
        if root is None:
            return 0
        return self.height(root.left) - self.height(root.right)
    def right_rotate(self, y):
        x = y.left
        t = x.right
        x.right = y
        y.left = t
        y.height = 1 + max(self.height(y.left), self.height(y.right))
        x.height = 1 + max(self.height(x.left), self.height(x.right))
        return x
    def left_rotate(self, x):
        y = x.right
        t = y.left
        y.left = x
        x.right = t
        x.height = 1 + max(self.height(x.left), self.height(x.right))
        y.height = 1 + max(self.height(y.left), self.height(y.right))
        return y
    def insert(self, root, data):
        if root is None:
            return Node(data)
        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)
```

```
        else:
            return root
        root.height = 1 + max(self.height(root.left), self.height(root.right))
        b = self.balance(root)
        if b > 1 and data < root.left.data:
            return self.right_rotate(root)
        if b < -1 and data > root.right.data:
            return self.left_rotate(root)
        if b > 1 and data > root.left.data:
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if b < -1 and data < root.right.data:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root
    def search(self, root, data):
        if root is None:
            return False
        if root.data == data:
            return True
        elif data < root.data:
            return self.search(root.left, data)
        else:
            return self.search(root.right, data)
    def find_min(self, root):
        current = root
        while current.left is not None:
            current = current.left
        return current
    def delete(self, root, data):
        if root is None:
            return root
        if data < root.data:
            root.left = self.delete(root.left, data)
        elif data > root.data:
            root.right = self.delete(root.right, data)
        else:
            if root.left is None:
                return root.right
            elif root.right is None:
                return root.left
            else:
                temp = self.find_min(root.right)
                root.data = temp.data
                root.right = self.delete(root.right, temp.data)
        root.height = 1 + max(self.height(root.left), self.height(root.right))
        b = self.balance(root)
        if b > 1 and self.balance(root.left) >= 0:
            return self.right_rotate(root)
        if b > 1 and self.balance(root.left) < 0:
```

```
            root.left = self.left_rotate(root.left)
            return self.right_rotate(root)
        if b < -1 and self.balance(root.right) <= 0:
            return self.left_rotate(root)
        if b < -1 and self.balance(root.right) > 0:
            root.right = self.right_rotate(root.right)
            return self.left_rotate(root)
        return root
    def inorder(self, root):
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)
    def preorder(self, root):
        if root:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)
    def postorder(self, root):
        if root:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")
    def level_order(self, root):
        if root is None:
            return
        queue = [root]
        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)

print("Ch. Jairamcharan...AV.SC.U4CSE25106...")
avl = AVLTree()
root = None

while True:
    print("\n\n--- AVL TREE ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Level Order Traversal")
    print("8. Exit")
```

```
    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter value to insert: "))
        root = avl.insert(root, data)
        print("Node inserted successfully.")
    elif choice == 2:
        data = int(input("Enter value to search: "))
        if avl.search(root, data):
            print("Element found.")
        else:
            print("Element not found.")
    elif choice == 3:
        data = int(input("Enter value to delete: "))
        root = avl.delete(root, data)
        print("Node deleted successfully.")
    elif choice == 4:
        print("Inorder Traversal:")
        avl.inorder(root)
    elif choice == 5:
        print("Preorder Traversal:")
        avl.preorder(root)
    elif choice == 6:
        print("Postorder Traversal:")
        avl.postorder(root)
    elif choice == 7:
        print("Level Order Traversal:")
        avl.level_order(root)
    elif choice == 8:
        print("Program terminated.")
        break
    else:
        print("Invalid choice.")
```

Output:

```
--- AVL TREE ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 6
Postorder Traversal:
2 9 5

--- AVL TREE ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 7
Level Order Traversal:
5 2 9

--- AVL TREE ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 8
Program terminated.
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106>
```

```
PS C:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106> python -u "c:\Users\jaira\OneDrive\Desktop\SEM-3\DSA\DSA25106\week-10\week10_1.py"
Ch. Jairamcharan...AV.SC.U4CSE25106...


--- AVL TREE ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter value to insert: 5
Node inserted successfully.


--- AVL TREE ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 1
Enter value to insert: 2
Node inserted successfully.


--- AVL TREE ---
1. Insert
2. Search
3. Delete
4. Inorder Traversal
5. Preorder Traversal
6. Postorder Traversal
7. Level Order Traversal
8. Exit
Enter your choice: 1
```

Error Table:

Error	Rectification
Invalid Input	Occurs when non-integer values are entered.

Description:

- AVL Tree is a self-balancing Binary Search Tree.
- Each node contains data, left child, right child and height.
- It maintains a balance factor of -1, 0 or +1 for every node.
- Insertion adds new nodes while maintaining the balance of the tree.
- Searching is used to find a particular element in the tree.
- Deletion removes a node and rebalances the tree using rotations.
- AVL Tree uses four rotations: LL, RR, LR and RL.
- It provides efficient searching, insertion and deletion operations.

Week-11

11)

Aim:

- Python program to implement Hashing (Separate chaining, Linear probing, and Quadratic probing).

Program:

```
size = 10
chain = [[] for i in range(size)]

def separate_chaining(key):
    index = key % size
    chain[index].append(key)

linear = [None] * size

def linear_probing(key):
    index = key % size
    while linear[index] is not None:
        index = (index + 1) % size
    linear[index] = key

quadratic = [None] * size

def quadratic_probing(key):
    index = key % size
    i = 0
    while i < size:
        new_index = (index + i * i) % size
        if quadratic[new_index] is None:
            quadratic[new_index] = key
            return
        i += 1
    print("Hash table is full")

n = int(input("Enter number of elements: "))
keys = []

for i in range(n):
    key = int(input("Enter key: "))
    keys.append(key)
    separate_chaining(key)
    linear_probing(key)
    quadratic_probing(key)

print("\nSeparate Chaining:")
for i in range(size):
```

```
    print(i, ":", chain[i])
print("\nLinear Probing:")
print(linear)
print("\nQuadratic Probing:")
print(quadratic)
```

Output:



Error Table:

Error	Rectification
Index Out of Range	Occurs when an invalid index is accessed.

Description:

- Hashing is a technique used to store and retrieve data efficiently using a hash function.
- A hash function calculates the index of a key in a hash table.
- Separate Chaining handles collisions by storing multiple elements at the same index using a list.
- Linear Probing resolves collisions by checking the next available position in the hash table.
- Quadratic Probing resolves collisions by checking positions using quadratic intervals.
- These three techniques help manage collisions and improve data storage and searching efficiency.